

Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский технологический университет «МИСИС»

Институт информационных технологий и компьютерных наук (ИТКН)
Кафедра инфокоммуникационных технологий (ИКТ)

Отчет по курсовой работе
по дисциплине «Клиент-серверные приложения и сетевые технологии»
на тему «Проектирование и реализация клиент-серверного приложения»
Вариант № 8

Выполнил:
студент группы БИСТ-24-1
Богданов Дмитрий Сергеевич
Подпись: _____

Проверил:
доцент каф. ИКТ,
Стучилин В.В.

Дата сдачи: «___» _____ 2026г.
Оценка: _____ Подпись: _____

Москва, 2026

Задание:

Разработать два программных модуля. Первый модуль генерирует данные по вершинам дерева в БД. Каждая вершина содержит номера дочерних узлов (максимум три дочерних узла). Наличие, или отсутствие дочернего узла должно определяться случайным образом. Второй модуль должен отображать сгенерированную информацию в виде графического дерева (горизонтального, или вертикального). Историю запросов необходимо сохранять в БД.

Принятые решения по реализации:

Компонент	Назначение
СУБД (SQLite / PostgreSQL 16)	Хранение метаданных деревьев, вершин дерева (с указанием родителя и номеров дочерних узлов) и журнала запросов. SQLite — для локальной разработки, PostgreSQL — при запуске через Docker Compose.
Сервер Laravel 13 (PHP 8.3)	Модуль 1: рекурсивная генерация случайного дерева в БД (TreeGenerationService). Модуль 2: чтение дерева из БД и сборка ответа в JSON (TreeResponseBuilder). Промежуточное ПО LogApiRequest сохраняет каждый запрос в таблицу request_logs.
Клиент iOS (SwiftUI)	Ввод параметров генерации (название, ориентация, глубина, вероятность, seed), запуск генерации, графическое отображение дерева в TreeCanvasView (вершины и направленные рёбра, ориентация вертикальная/горизонтальная), просмотр истории деревьев и журнала запросов.

Структура системы (топология сети с указанием сетевых протоколов):

Серверная часть представляет собой REST-приложение, написанное на фреймворке Laravel 13 (PHP 8.3). Хранение данных осуществляется в реляционной СУБД: по умолчанию используется SQLite (файл database/database.sqlite), при запуске через Docker Compose поднимается PostgreSQL 16. Клиентом является мобильное приложение на iOS (SwiftUI),

которое обращается к серверу по протоколу HTTP в формате JSON. При работе в симуляторе iOS используется адрес 127.0.0.1, при запуске на физическом устройстве — IP-адрес компьютера в локальной сети Wi-Fi.

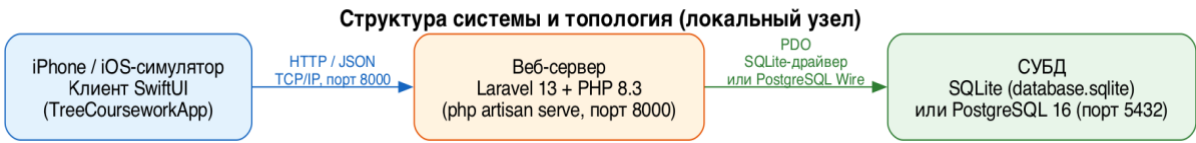


Рисунок 1 — Структура системы: iOS-клиент общается с сервером Laravel по HTTP/JSON, сервер использует PDO для работы с СУБД.

Используемые сетевые протоколы

Протокол	Уровень / назначение
HTTP/1.1	Прикладной уровень: REST-запросы клиента к API Laravel (GET /api/trees, GET /api/trees/latest, GET /api/trees/{id}, POST /api/trees/generate, GET /api/request-logs). Тело запроса и ответа — JSON.
TCP	Транспортный уровень: надёжная доставка сегментов между клиентом и портом веб-сервера (8000 для php artisan serve).
IP	Сетевой уровень: адресация узлов (127.0.0.1 — в симуляторе iOS; IP компьютера в локальной Wi-Fi — для запуска на iPhone).
PostgreSQL Wire Protocol	Взаимодействие PHP-приложения с СУБД при работе в Docker (порт 5432). Для локального запуска используется драйвер SQLite (файл database.sqlite).

Состав программных модулей:

Сервер (Laravel):

routes/api.php — описание HTTP-маршрутов API.

app/Models/Tree.php, TreeNode.php, RequestLog.php — Eloquent-модели таблиц trees, tree_nodes, request_logs.

app/Http/Controllers/Api/TreeController.php — контроллер дерева: списки, получение по id, последнее дерево, генерация нового дерева.

app/Http/Controllers/Api/RequestLogController.php — контроллер журнала запросов.

app/Http/Middleware/LogApiRequest.php — промежуточное ПО, сохраняющее каждый HTTP-запрос к API в таблицу request_logs (модуль журналирования).

app/Services/TreeGenerationService.php — модуль 1: рекурсивная генерация дерева в БД с использованием псевдослучайного генератора Mt19937, инициализируемого зерном seed.

app/Services/TreeResponseBuilder.php — модуль 2: чтение вершин дерева из БД и формирование вложенной JSON-структуры для клиента.

database/migrations/2026_04_13_101332_create_trees_table.php — схема таблицы trees.

database/migrations/2026_04_13_101333_create_tree_nodes_table.php — схема таблицы tree_nodes.

database/migrations/2026_04_13_101334_create_request_logs_table.php — схема таблицы request_logs.

database/migrations/2026_04_13_140000_add_child_numbers_to_tree_nodes_table.php — добавление JSON-колонки child_numbers (номера дочерних узлов).

bootstrap/app.php — регистрация LogApiRequest как api-middleware.

Клиент (iOS, SwiftUI):

TreeCourseworkApp.swift — точка входа в приложение.

APIClient.swift — слой работы с REST API (URLSession, JSON-кодирование/декодирование с конвертацией snake_case ↔ camelCase).

Models.swift — модели DTO: TreeSummary, TreeRecord, TreeNode, RequestLogEntry, GenerateTreeRequest.

AppViewModel.swift — управление состоянием (ObservableObject): параметры генерации, текущее дерево, история деревьев, журнал запросов.

ContentView.swift — основной экран: панели «Laravel API», «Генерация дерева», «Список деревьев», «История запросов», «Текущее дерево».

TreeCanvasView.swift — отрисовка дерева в виде ориентированного графа (вершины-бейджи и направленные стрелки).

TreeGraphDetailView.swift — полноэкранный просмотр графа с переключателем ориентации.

ThemeColors.swift — централизованные цвета интерфейса.

Блок-схемы:

Блок-схема: обработка HTTP-запроса в Laravel

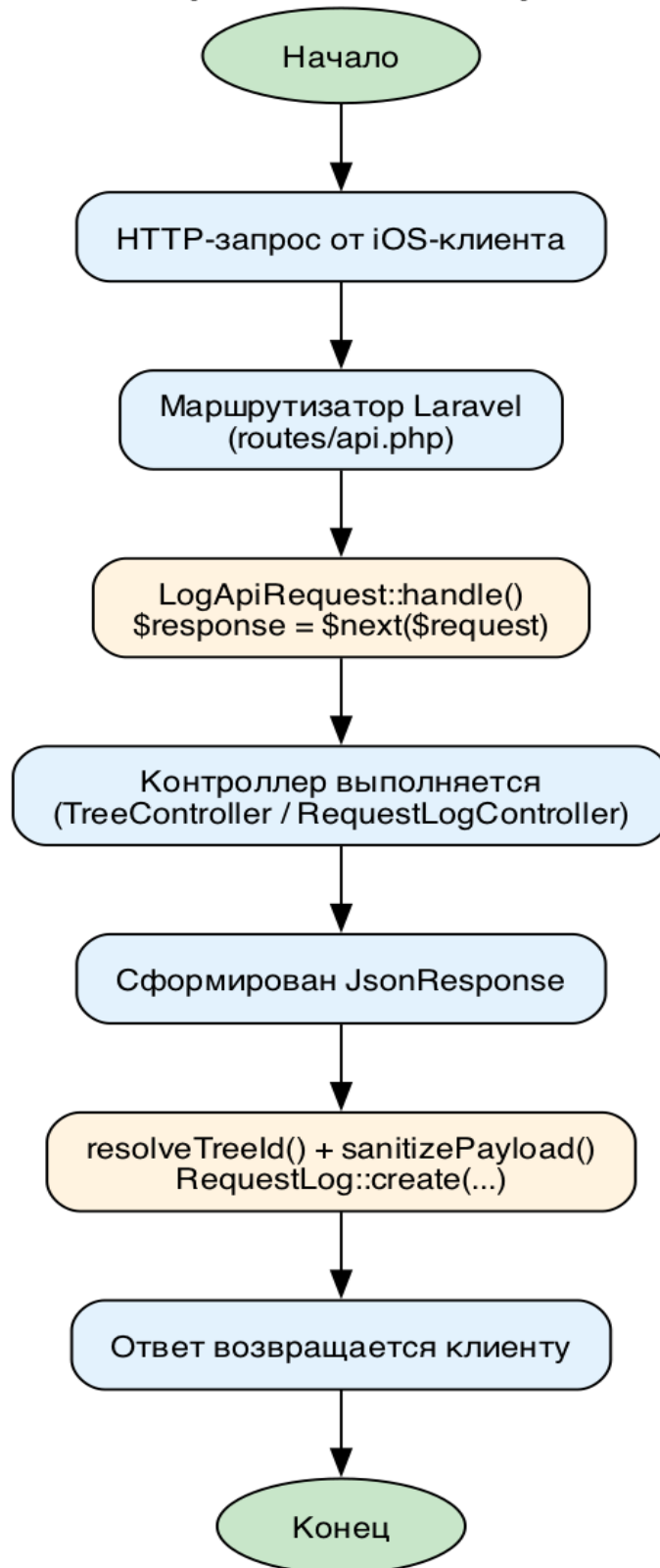


Рисунок 2 — Блок-схема: общая обработка HTTP-запроса (Laravel + middleware LogApiRequest).

Блок-схема: POST /api/trees/generate (TreeController::store)

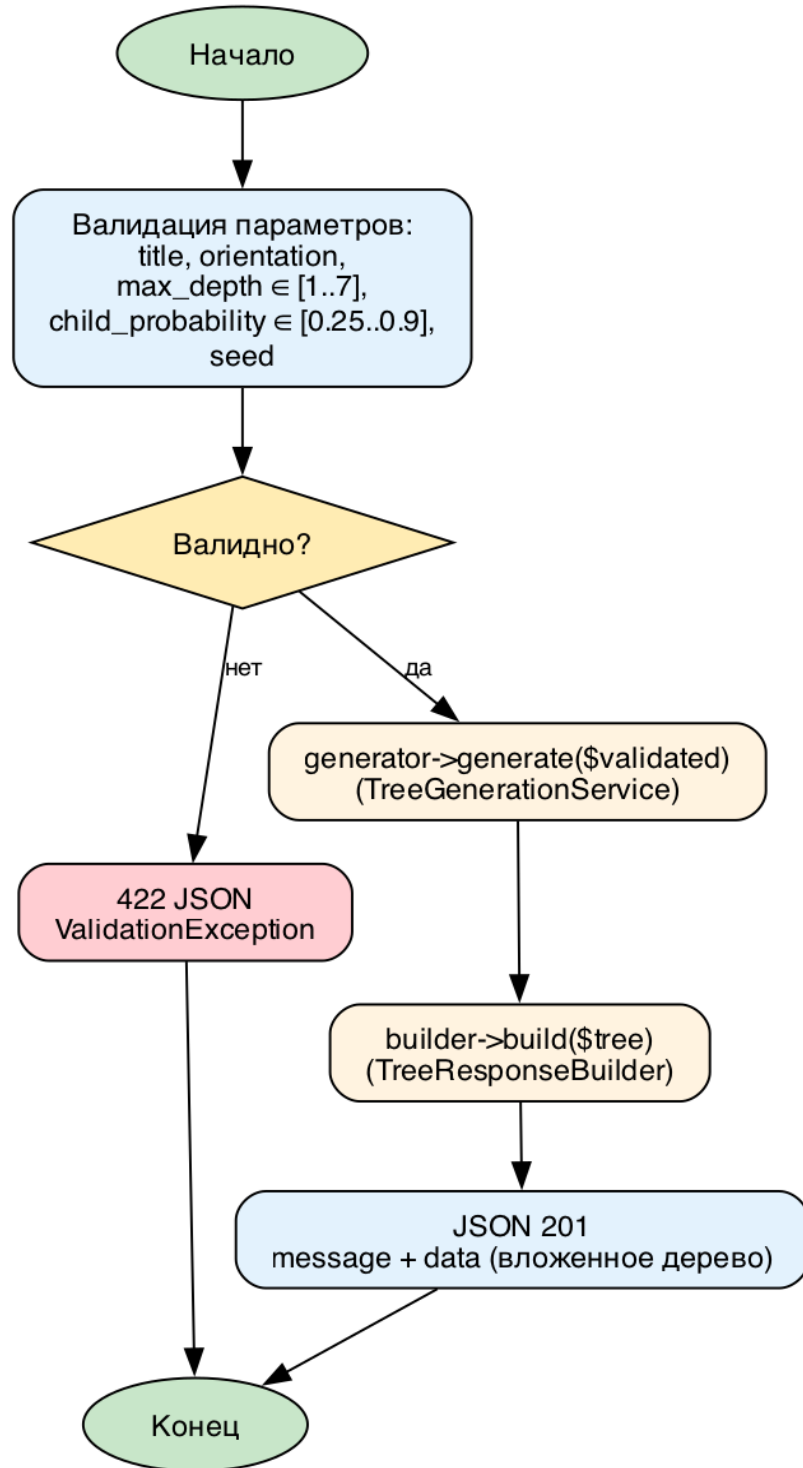


Рисунок 3 — Блок-схема: POST /api/trees/generate (TreeController::store).

Блок-схема: алгоритм генерации (TreeGenerationService) — модуль 1

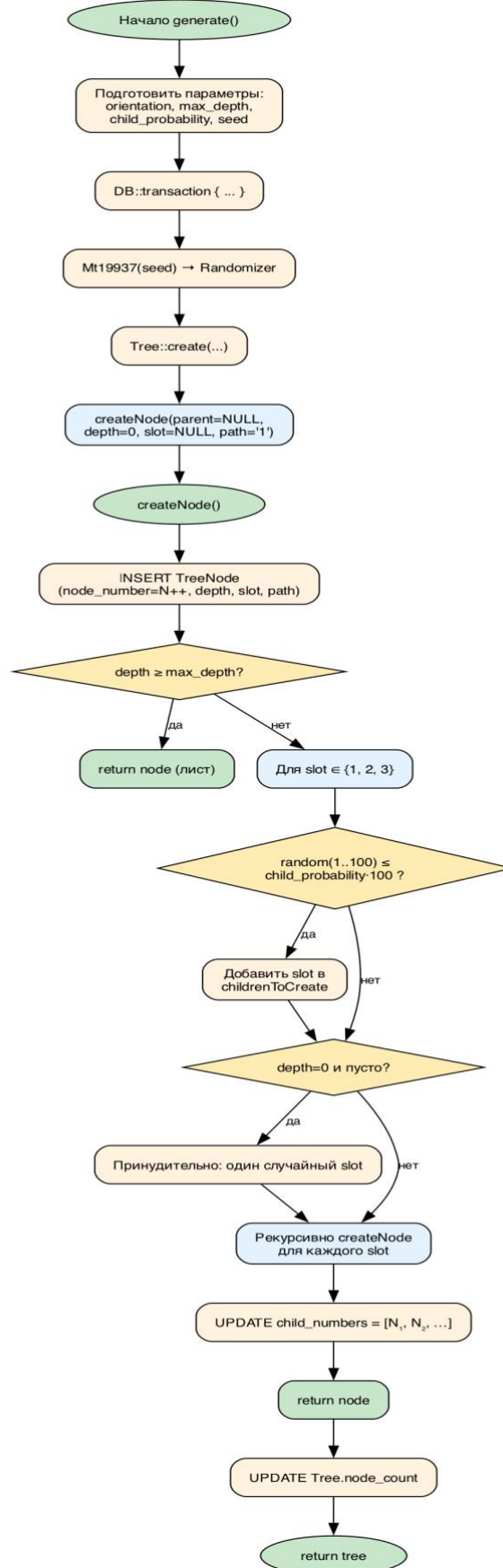


Рисунок 4 — Блок-схема: алгоритм рекурсивной генерации дерева (TreeGenerationService) — модуль 1.

Блок-схема: TreeResponseBuilder::build (модуль 2 — чтение дерева)

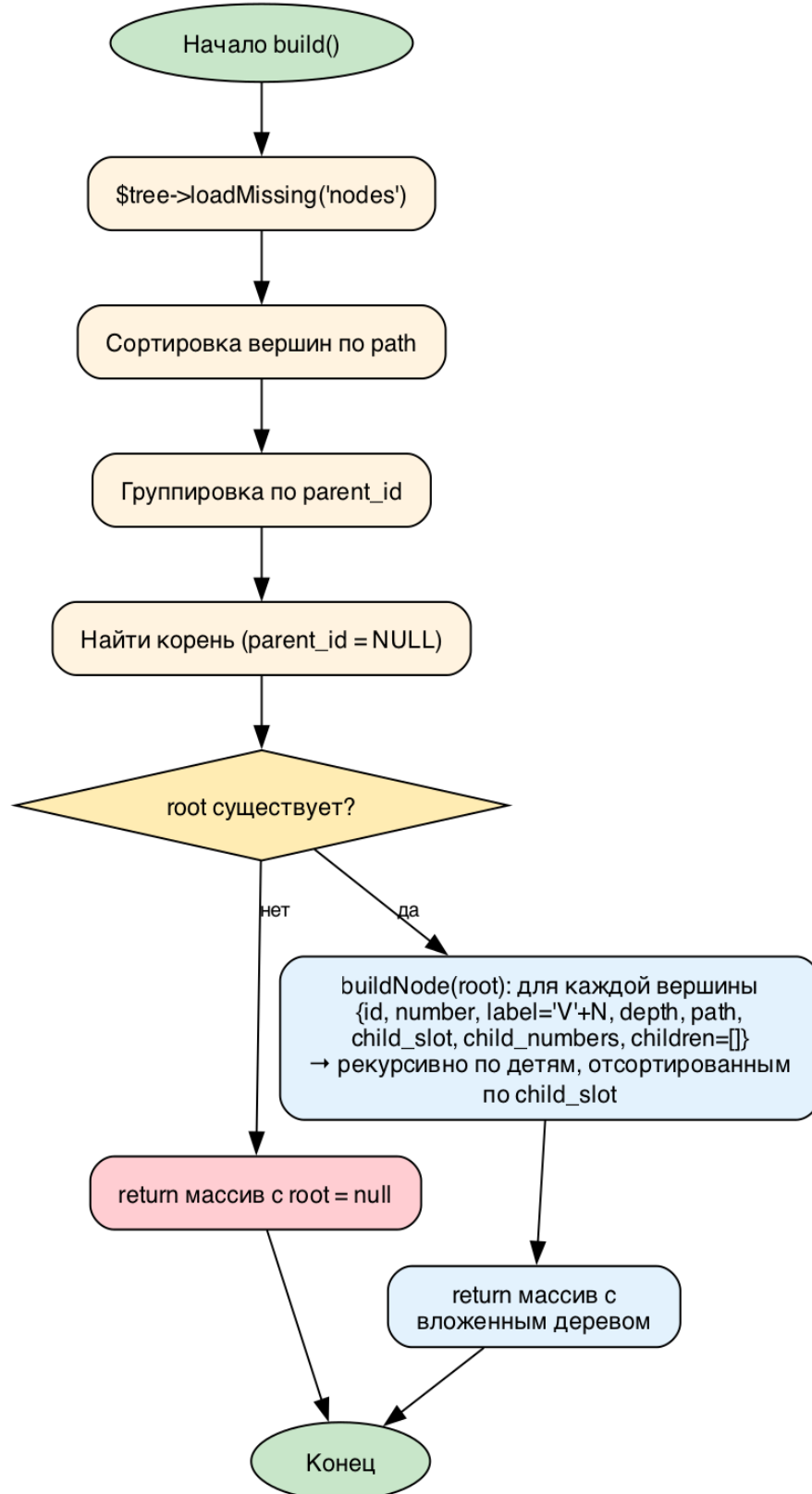


Рисунок 5 — Блок-схема: чтение дерева из БД и сборка JSON (TreeResponseBuilder) — модуль 2.

Блок-схема: LogApiRequest (журнал запросов)

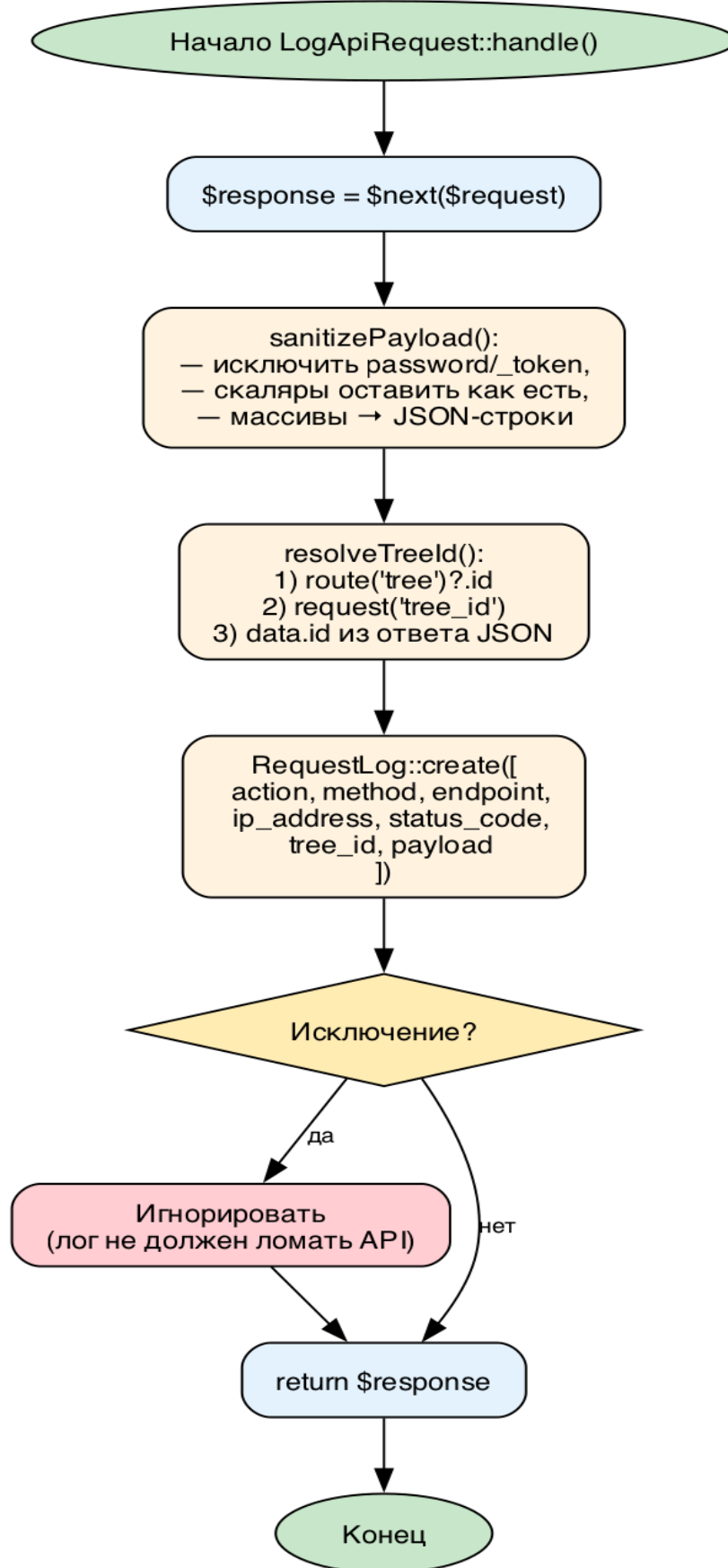


Рисунок 6 — Блок-схема: сохранение запроса в БД (LogApiRequest::handle).

Блок-схема: клиентский сценарий «Сгенерировать дерево»

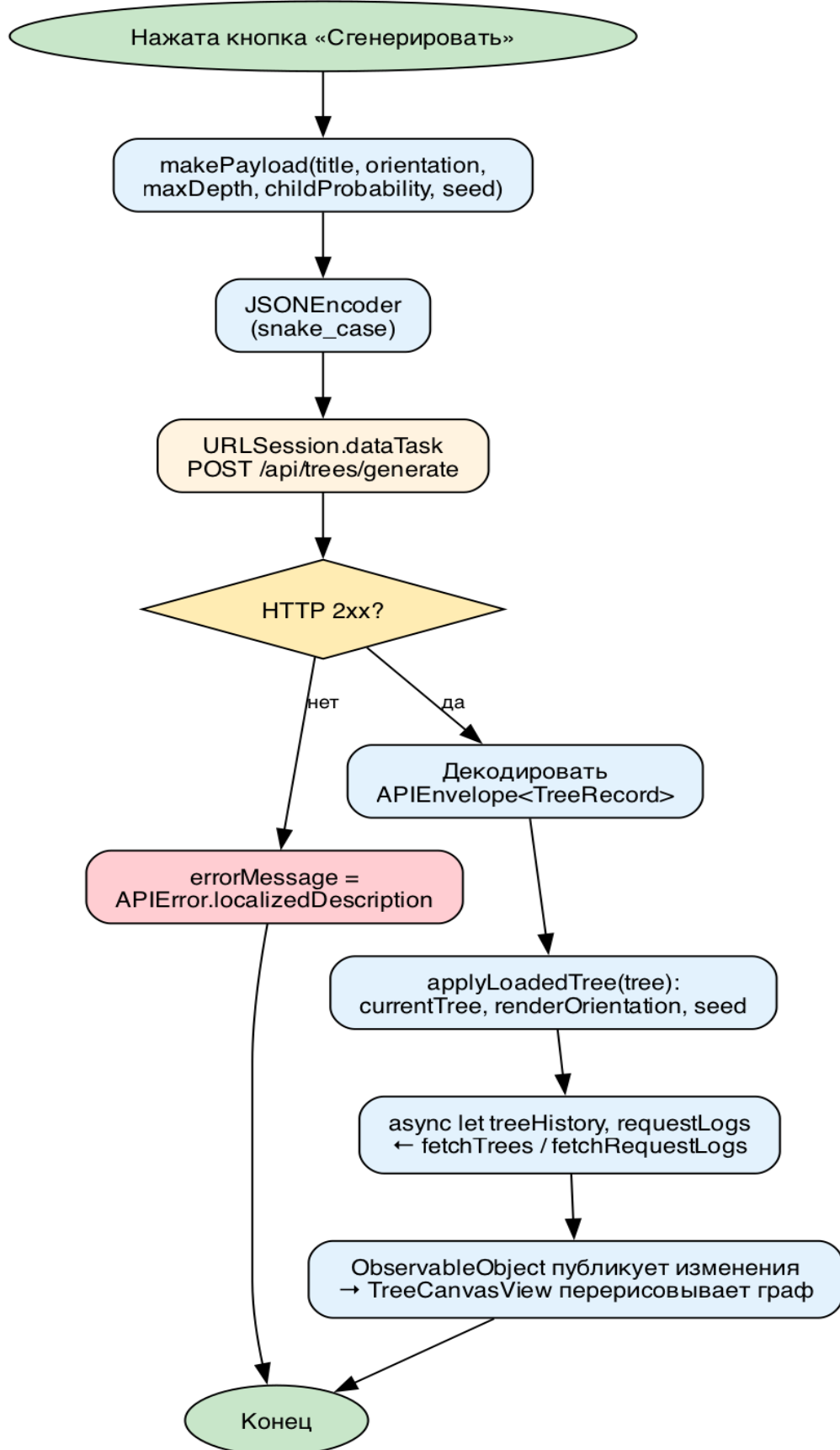


Рисунок 7 — Блок-схема: клиентский сценарий «Сгенерировать дерево» (AppViewModel + APIClient).

Структура БД:

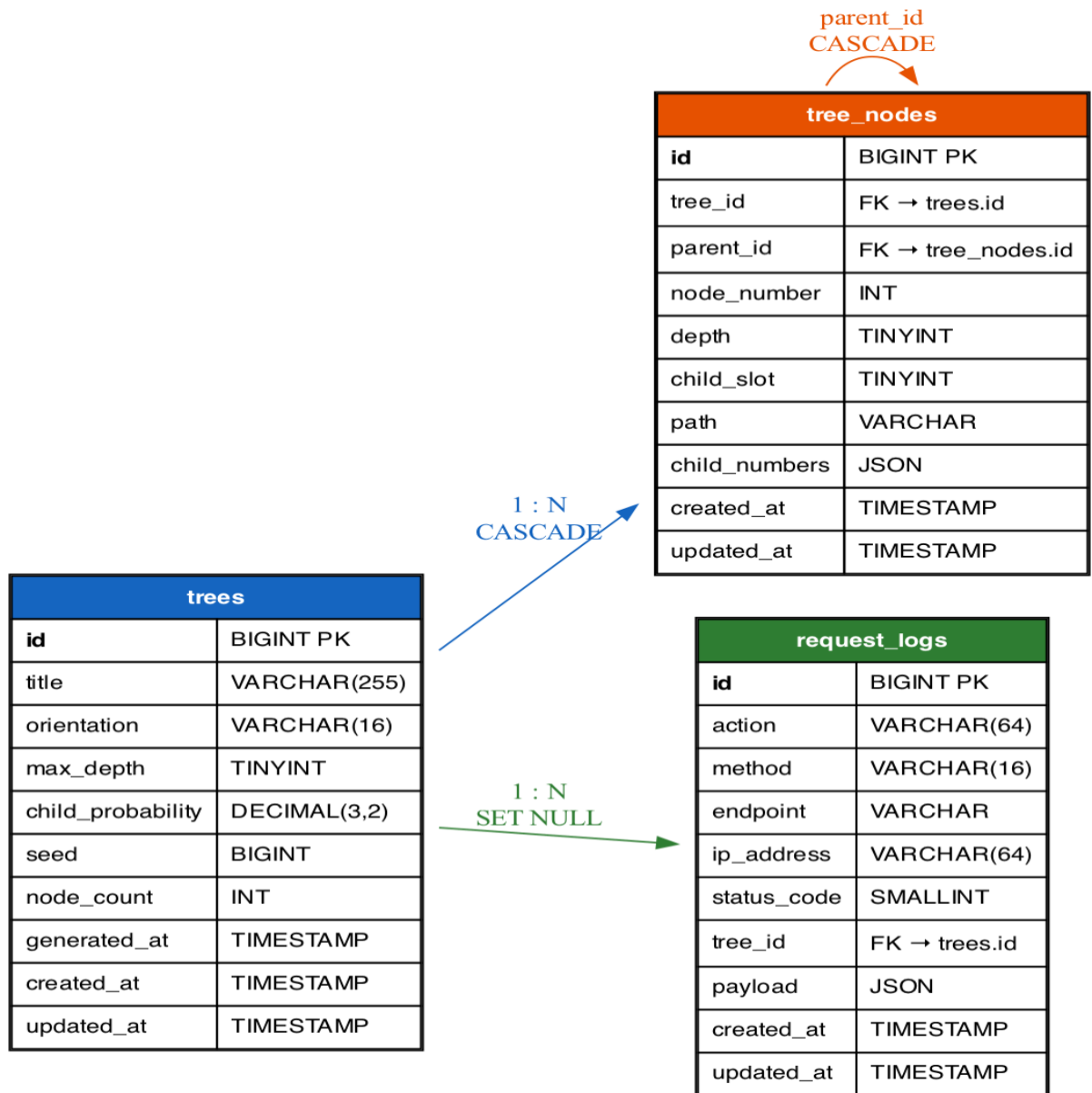


Рисунок 8 — ER-диаграмма базы данных: таблицы *trees*, *tree_nodes* (с самоссылкой *parent_id*) и *request_logs*.

База данных *tree_coursework* (PostgreSQL) или локальный файл *database.sqlite*.

Кодировка: UTF-8.

Движок: InnoDB-совместимый (для MySQL) либо стандартный (PostgreSQL / SQLite).

Таблица *trees*

Поле	Тип	Назначение
id	BIGINT UNSIGNED, PK, AI	Идентификатор дерева
title	VARCHAR(255), NULLABLE	Название дерева, заданное пользователем
orientation	VARCHAR(16), DEFAULT 'vertical'	Ориентация отображения: vertical или horizontal
max_depth	TINYINT UNSIGNED, NOT NULL	Максимальная глубина дерева (1–7)
child_probability	DECIMAL(3,2), DEFAULT 0.60	Вероятность создания каждого дочернего узла (0.25–0.90)
seed	BIGINT UNSIGNED, NOT NULL	Зерно генератора псевдослучайных чисел (Mt19937) — обеспечивает воспроизводимость
node_count	INT UNSIGNED, DEFAULT 0	Итоговое количество вершин в дереве
generated_at	TIMESTAMP, NOT NULL	Время генерации дерева
created_at, updated_at	TIMESTAMP, AUTO	Служебные временные метки Laravel

Таблица tree_nodes

Поле	Тип	Назначение
id	BIGINT UNSIGNED, PK, AI	Идентификатор вершины
tree_id	BIGINT UNSIGNED, FK → trees(id), CASCADE	Дерево, которому принадлежит вершина
parent_id	BIGINT UNSIGNED, NULL, FK → tree_nodes(id), CASCADE	Родительская вершина (NULL — корень)
node_number	INT UNSIGNED, UNIQUE(tree_id, node_number)	Порядковый номер вершины в дереве (1, 2, 3, ...)
depth	TINYINT UNSIGNED, NOT NULL	Уровень вложенности (0 — корень)
child_slot	TINYINT UNSIGNED, NULL	Позиция вершины у родителя: 1, 2 или 3

Поле	Тип	Назначение
path	VARCHAR, UNIQUE(tree_id, path)	Точечный путь от корня (например, «1.2.1») для быстрого поиска поддеревьев
child_numbers	JSON, NULL	Массив номеров дочерних узлов (например, [2, 3]) — соответствует требованию задания
created_at, updated_at	TIMESTAMP, AUTO	Служебные временные метки Laravel

Таблица request_logs

Поле	Тип	Назначение
id	BIGINT UNSIGNED, PK, AI	Номер записи журнала
action	VARCHAR(64), NOT NULL	Имя маршрута Laravel (trees.index, trees.latest, trees.generate, ...)
method	VARCHAR(16), NOT NULL	HTTP-метод запроса (GET, POST)
endpoint	VARCHAR, NOT NULL	Путь запроса (например, /api/trees/generate)
ip_address	VARCHAR(64), NULL	IP клиента
status_code	SMALLINT UNSIGNED, NOT NULL	Код ответа HTTP (200, 201, 404, 500)
tree_id	BIGINT UNSIGNED, NULL, FK → trees(id), SET NULL	Связанное дерево (если применимо)
payload	JSON, NULL	Тело запроса (без секретов)
created_at, updated_at	TIMESTAMP, AUTO	Время регистрации запроса

Пример выполнения программы:

Серверная часть запускается командой `php artisan serve --host=127.0.0.1 --port=3032` в каталоге backend, после чего на iOS-симуляторе разворачивается клиентское приложение. Ниже приведены реальные снимки экрана работы программы.

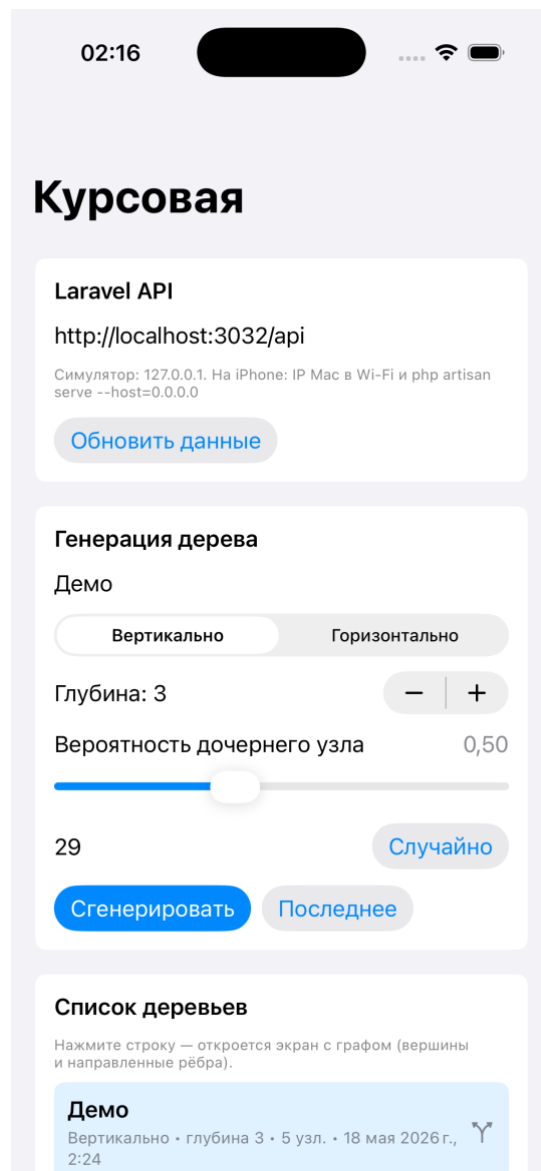


Рисунок 9 — Главный экран iOS-приложения: панели «Laravel API», «Генерация дерева» (название, ориентация, глубина, вероятность дочернего узла, seed, кнопки «Случайно», «Сгенерировать», «Последнее») и начало списка деревьев.

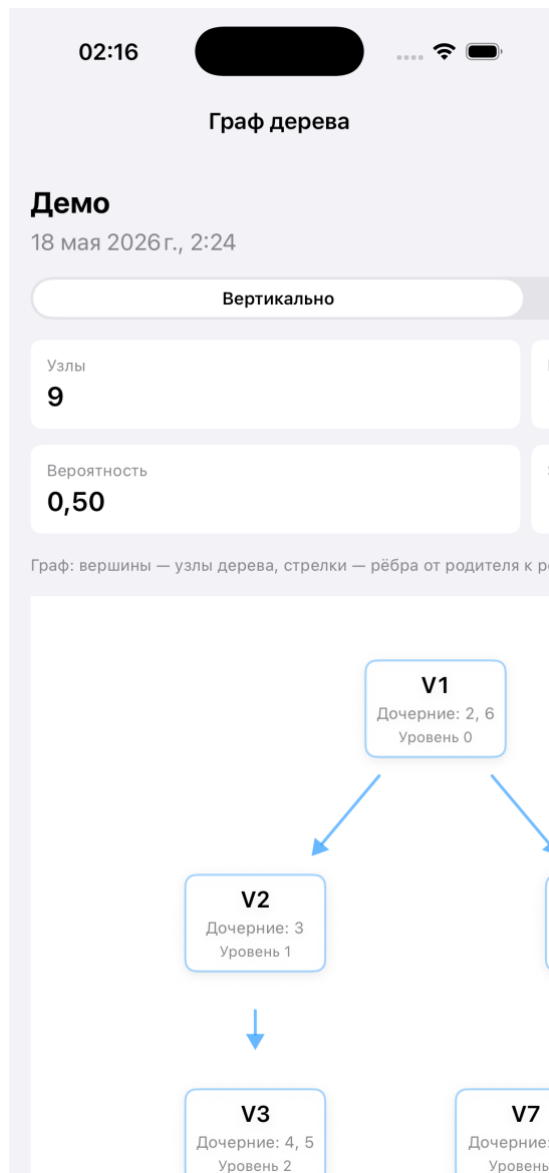


Рисунок 10 — Полноэкранный экран *TreeGraphDetailView*: вертикальный граф дерева *id=14* (9 вершин). Корень *V1* имеет потомков *V2* и *V6*, у каждой вершины подписаны номера дочерних узлов и уровень.

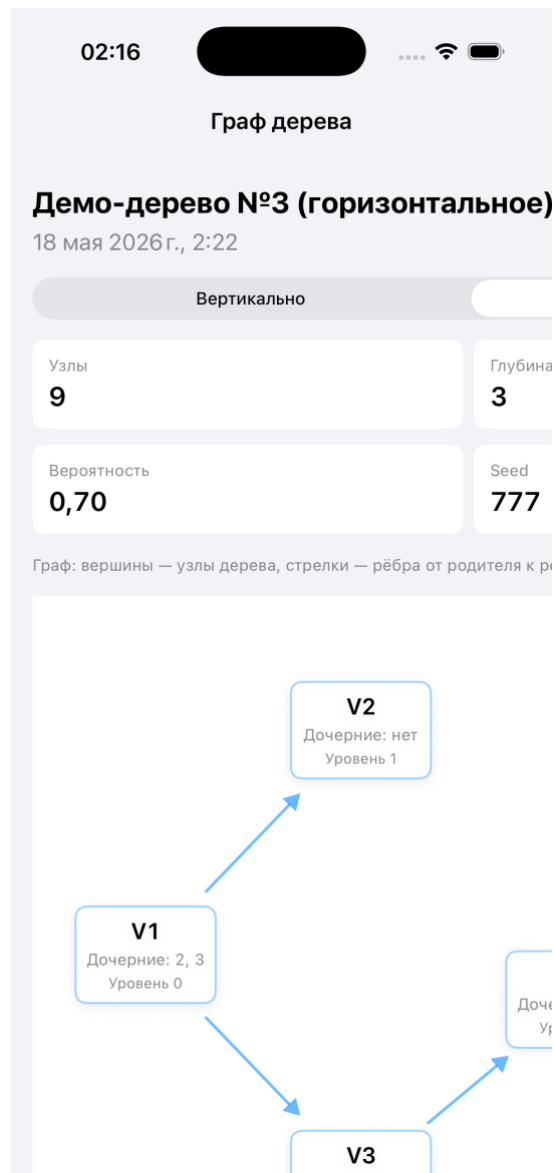


Рисунок 11 — Тот же экран в горизонтальной ориентации (дерево $id=12$, $seed=777$): корень $V1$ слева, потомки $V2$ (без дочерних) и $V3$ (с дочерними 4, 5) растут вправо.

id	action	method	endpoint	ip	code	tree_id	time
130	trees.latest	GET	/api/trees/latest	127.0.0.1	200	26	2026-05-18 02:26:02
129	request-logs.index	GET	/api/request-logs	127.0.0.1	200		2026-05-18 02:26:02
128	trees.index	GET	/api/trees	127.0.0.1	200		2026-05-18 02:26:02
127	request-logs.index	GET	/api/request-logs	127.0.0.1	200		2026-05-18 02:25:05
126	trees.show	GET	/api/trees/12	127.0.0.1	200	12	2026-05-18 02:25:05
125	request-logs.index	GET	/api/request-logs	127.0.0.1	200		2026-05-18 02:24:43
124	trees.show	GET	/api/trees/14	127.0.0.1	200	14	2026-05-18 02:24:43
123	trees.generate	POST	/api/trees/generate	127.0.0.1	201	26	2026-05-18 02:24:13
122	trees.generate	POST	/api/trees/generate	127.0.0.1	201	25	2026-05-18 02:24:13
121	trees.generate	POST	/api/trees/generate	127.0.0.1	201	24	2026-05-18 02:24:13
120	trees.generate	POST	/api/trees/generate	127.0.0.1	201	23	2026-05-18 02:24:13
119	trees.generate	POST	/api/trees/generate	127.0.0.1	201	22	2026-05-18 02:24:13
118	trees.generate	POST	/api/trees/generate	127.0.0.1	201	21	2026-05-18 02:24:13
117	trees.generate	POST	/api/trees/generate	127.0.0.1	201	20	2026-05-18 02:24:13
116	trees.generate	POST	/api/trees/generate	127.0.0.1	201	19	2026-05-18 02:24:13
115	trees.generate	POST	/api/trees/generate	127.0.0.1	201	18	2026-05-18 02:24:13
114	trees.generate	POST	/api/trees/generate	127.0.0.1	201	17	2026-05-18 02:24:13
113	trees.generate	POST	/api/trees/generate	127.0.0.1	201	16	2026-05-18 02:24:13
112	trees.generate	POST	/api/trees/generate	127.0.0.1	201	15	2026-05-18 02:24:13
111	trees.generate	POST	/api/trees/generate	127.0.0.1	201	14	2026-05-18 02:24:13

Рисунок 12 — Содержимое таблицы request_logs в SQLite: видны записи о генерации деревьев (POST /api/trees/generate, статус 201), просмотре дерева (GET /api/trees/14, 200), получении последнего (GET /api/trees/latest) и журнала запросов.

id	tree_id	parent_id	node_number	depth	child_slot	path	child_numbers
226	14		1	0		1	[2,6]
227	14	226	2	1	1	1.1	[3]
228	14	227	3	2	1	1.1.1	[4,5]
229	14	228	4	3	1	1.1.1.1	[]
230	14	228	5	3	2	1.1.1.2	[]
231	14	226	6	1	3	1.3	[7,8]
232	14	231	7	2	1	1.3.1	[]
233	14	231	8	2	3	1.3.3	[9]
234	14	233	9	3	2	1.3.3.2	[]

Рисунок 13 — Содержимое таблицы tree_nodes для дерева id=14: 9 вершин с указанием родителя (parent_id), номера (node_number), глубины (depth), позиции у родителя (child_slot), точечного пути (path) и JSON-массива номеров дочерних узлов (child_numbers).

Листинг программы:

routes/api.php

```
<?php
```

```
use App\Http\Controllers\Api\RequestLogController;
use App\Http\Controllers\Api\TreeController;
```

```

use Illuminate\Support\Facades\Route;

Route::get('/trees', [TreeController::class, 'index'])->name('trees.index');
Route::get('/trees/latest', [TreeController::class, 'latest'])->name('trees.latest');
Route::get('/trees/{tree}', [TreeController::class, 'show'])->name('trees.show');
Route::post('/trees/generate', [TreeController::class, 'store'])->
>name('trees.generate');

Route::get('/request-logs', [RequestLogController::class, 'index'])->name('request-
logs.index');

```

app/Http/Controllers/Api/TreeController.php

```
<?php
```

```

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Tree;
use App\Services\TreeGenerationService;
use App\Services\TreeResponseBuilder;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class TreeController extends Controller
{
    public function index(Request $request): JsonResponse
    {
        $limit = min((int) $request->integer('limit', 10), 50);

        $trees = Tree::query()
            ->orderByDesc('generated_at')
            ->orderByDesc('id')
            ->limit($limit)
            ->get()
            ->map(fn (Tree $tree) => [
                'id' => $tree->id,
                'title' => $tree->title,
                'orientation' => $tree->orientation,
                'max_depth' => $tree->max_depth,
                'child_probability' => $tree->child_probability,
                'seed' => $tree->seed,
                'node_count' => $tree->node_count,
                'generated_at' => $tree->generated_at?->toIso8601String(),
            ]);

        return response()->json(['data' => $trees]);
    }
}

```

```

public function latest(TreeResponseBuilder $builder): JsonResponse
{
    $tree = Tree::query()
        ->orderByDesc('generated_at')
        ->orderByDesc('id')
        ->firstOrFail();

    return response()->json(['data' => $builder->build($tree)]);
}

public function show(Tree $tree, TreeResponseBuilder $builder): JsonResponse
{
    return response()->json(['data' => $builder->build($tree)]);
}

public function store(
    Request $request,
    TreeGenerationService $generator,
    TreeResponseBuilder $builder
): JsonResponse {
    $validated = $request->validate([
        'title' => ['nullable', 'string', 'max:255'],
        'orientation' => ['nullable', 'in:vertical,horizontal'],
        'max_depth' => ['nullable', 'integer', 'min:1', 'max:7'],
        'child_probability' => ['nullable', 'numeric', 'min:0.25', 'max:0.9'],
        'seed' => ['nullable', 'integer', 'min:1'],
    ]);

    $tree = $generator->generate($validated);

    return response()->json([
        'message' => 'Дерево успешно сгенерировано.',
        'data' => $builder->build($tree),
    ], 201);
}
}

```

app/Http/Controllers/Api/RequestLogController.php

```
<?php
```

```

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\RequestLog;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class RequestLogController extends Controller

```

```

{
    public function index(Request $request): JsonResponse
    {
        $limit = min((int) $request->integer('limit', 25), 100);

        $logs = RequestLog::query()
            ->latest()
            ->with('tree:id,title')
            ->limit($limit)
            ->get()
            ->map(fn (RequestLog $log) => [
                'id' => $log->id,
                'action' => $log->action,
                'method' => $log->method,
                'endpoint' => $log->endpoint,
                'ip_address' => $log->ip_address,
                'status_code' => $log->status_code,
                'tree' => $log->tree ? ['id' => $log->tree->id, 'title' => $log->tree->title] : null,
                'payload' => $log->payload,
                'created_at' => $log->created_at?->toIso8601String(),
            ]);

        return response()->json(['data' => $logs]);
    }
}

```

app/Http/Middleware/LogApiRequest.php

```

<?php

namespace App\Http\Middleware;

use App\Models\RequestLog;
use Closure;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;
use Throwable;

class LogApiRequest
{
    public function handle(Request $request, Closure $next): Response
    {
        $response = $next($request);

        try {
            RequestLog::query()->create([
                'action' => $request->route()?->getName() ?? $request->path(),
                'method' => $request->method(),
            ]);
        } catch (Throwable $e) {
            //
        }

        return $response;
    }
}

```

```

        'endpoint' => '/' . $request->path(),
        'ip_address' => $request->ip(),
        'status_code' => $response->getStatusCode(),
        'tree_id' => $this->resolveTreeId($request, $response),
        'payload' => $this->sanitizePayload($request),
    ]);
} catch (Throwable) {
    // Лог запроса не должен ломать основной сценарий приложения.
}

return $response;
}

private function sanitizePayload(Request $request): ?array
{
    $payload = collect($request->except(['password', 'password_confirmation',
'_token']))
        ->map(function (mixed $value) {
            if (is_scalar($value) || is_null($value)) {
                return $value;
            }
            return json_encode($value, JSON_UNESCAPED_UNICODE);
        })
        ->all();

    return $payload === [] ? null : $payload;
}

private function resolveTreeId(Request $request, Response $response): ?int
{
    $routeTree = $request->route('tree');
    if ($routeTree?->id) {
        return (int) $routeTree->id;
    }

    $requestTreeId = $request->integer('tree_id');
    if ($requestTreeId > 0) {
        return $requestTreeId;
    }

    if (! $response instanceof JsonResponse) {
        return null;
    }

    $payload = $response->getData(true);
    $responseTreeId = data_get($payload, 'data.id');

    return is_numeric($responseTreeId) ? (int) $responseTreeId : null;
}
}

```

app/Services/TreeGenerationService.php (модуль 1)

```
<?php

namespace App\Services;

use App\Models\Tree;
use App\Models\TreeNode;
use Illuminate\Support\Carbon;
use Illuminate\Support\Facades\DB;
use Random\Engine\Mt19937;
use Random\Randomizer;

class TreeGenerationService
{
    private int $nextNodeNumber = 1;

    public function generate(array $payload): Tree
    {
        $orientation = $payload['orientation'] ?? 'vertical';
        $maxDepth = (int) ($payload['max_depth'] ?? 4);
        $childProbability = round((float) ($payload['child_probability'] ?? 0.6), 2);
        $seed = (int) ($payload['seed'] ?? random_int(1, PHP_INT_MAX));

        return DB::transaction(function () use ($payload, $orientation, $maxDepth,
        $childProbability, $seed) {
            $randomizer = new Randomizer(new Mt19937($seed));

            $tree = Tree::query()->create([
                'title' => $payload['title'] ?? 'Сгенерированное дерево',
                'orientation' => $orientation,
                'max_depth' => $maxDepth,
                'child_probability' => $childProbability,
                'seed' => $seed,
                'generated_at' => Carbon::now(),
            ]);

            $this->nextNodeNumber = 1;
            $this->createNode(
                tree: $tree,
                parent: null,
                depth: 0,
                childSlot: null,
                path: '1',
                randomizer: $randomizer,
            );

            $tree->forceFill(['node_count' => $this->nextNodeNumber - 1])->save();
```

```

        return $tree->fresh('nodes');
    });
}

private function createNode(
    Tree $tree, ?TreeNode $parent, int $depth,
    ?int $childSlot, string $path, Randomizer $randomizer
): TreeNode {
    $node = $tree->nodes()->create([
        'parent_id' => $parent?->id,
        'node_number' => $this->nextNodeNumber++,
        'depth' => $depth,
        'child_slot' => $childSlot,
        'path' => $path,
        'child_numbers' => [],
    ]);

    if ($depth >= $tree->max_depth) {
        return $node;
    }

    $childrenToCreate = collect([1, 2, 3])
        ->filter(fn () => $this->shouldCreateChild($tree->child_probability,
$randomizer))
        ->values();

    if ($depth === 0 && $childrenToCreate->isEmpty()) {
        $childrenToCreate = collect([$randomizer->getInt(1, 3)]);
    }

    $children = [];
    foreach ($childrenToCreate as $slot) {
        $children[] = $this->createNode(
            tree: $tree, parent: $node, depth: $depth + 1,
            childSlot: $slot, path: "{$path}.{$slot}", randomizer: $randomizer,
        );
    }

    $node->forceFill([
        'child_numbers' => array_map(
            static fn (TreeNode $child): int => $child->node_number, $children
        ),
    ])->save();

    return $node;
}

private function shouldCreateChild(float $probability, Randomizer $randomizer):
bool

```



```

    {
        return $randomizer->getInt(1, 100) <= (int) round($probability * 100);
    }
}

```

app/Services/TreeResponseBuilder.php (модуль 2)

```
<?php
```

```
namespace App\Services;
```

```

use App\Models\Tree;
use App\Models\TreeNode;
use Illuminate\Support\Collection;

```

```

class TreeResponseBuilder
{
    public function build(Tree $tree): array
    {
        $tree->loadMissing('nodes');

        $nodes = $tree->nodes->sortBy('path')->values();
        $groupedByParent = $nodes->groupBy(fn (TreeNode $node) => $node->parent_id ??
'root');
        $root = $nodes->firstWhere('parent_id', null);

        return [
            'id' => $tree->id,
            'title' => $tree->title,
            'orientation' => $tree->orientation,
            'max_depth' => $tree->max_depth,
            'child_probability' => $tree->child_probability,
            'seed' => $tree->seed,
            'node_count' => $tree->node_count,
            'generated_at' => $tree->generated_at?->toIso8601String(),
            'root' => $root ? $this->buildNode($root, $groupedByParent) : null,
        ];
    }
}

```

```

private function buildNode(TreeNode $node, Collection $groupedByParent): array
{
    $children = $groupedByParent
        ->get($node->id, collect())
        ->sortBy('child_slot')
        ->values();

    return [
        'id' => $node->id,
        'number' => $node->node_number,
        'label' => 'V'.$node->node_number,
    ];
}

```

```

        'depth' => $node->depth,
        'path' => $node->path,
        'child_slot' => $node->child_slot,
        'child_numbers' => $node->child_numbers ?? $children-
>pluck('node_number')->all(),
        'children' => $children
            ->map(fn (TreeNode $child) => $this->buildNode($child,
$groupedByParent))
            ->all(),
    ];
}
}

```

database/migrations/*_create_trees_table.php

```
<?php
```

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(): void
    {
        Schema::create('trees', function (Blueprint $table) {
            $table->id();
            $table->string('title')->nullable();
            $table->string('orientation', 16)->default('vertical');
            $table->unsignedTinyInteger('max_depth');
            $table->decimal('child_probability', 3, 2)->default(0.60);
            $table->unsignedBigInteger('seed');
            $table->unsignedInteger('node_count')->default(0);
            $table->timestamp('generated_at');
            $table->timestamps();
        });
    }

    public function down(): void { Schema::dropIfExists('trees'); }
};

```

database/migrations/*_create_tree_nodes_table.php

```
<?php
```

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {

```

```

public function up(): void
{
    Schema::create('tree_nodes', function (Blueprint $table) {
        $table->id();
        $table->foreignId('tree_id')->constrained()->cascadeOnDelete();
        $table->foreignId('parent_id')->nullable()->constrained('tree_nodes')->
>cascadeOnDelete();
        $table->unsignedInteger('node_number');
        $table->unsignedTinyInteger('depth');
        $table->unsignedTinyInteger('child_slot')->nullable();
        $table->string('path');
        $table->timestamps();

        $table->unique(['tree_id', 'node_number']);
        $table->unique(['tree_id', 'path']);
    });

    // В отдельной миграции добавляется JSON-колонка child_numbers:
    // $table->json('child_numbers')->nullable()->after('path');
}

public function down(): void { Schema::dropIfExists('tree_nodes'); }
};

```

database/migrations/*_create_request_logs_table.php

```

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(): void
    {
        Schema::create('request_logs', function (Blueprint $table) {
            $table->id();
            $table->string('action', 64);
            $table->string('method', 16);
            $table->string('endpoint');
            $table->string('ip_address', 64)->nullable();
            $table->unsignedSmallInteger('status_code');
            $table->foreignId('tree_id')->nullable()->constrained()->nullOnDelete();
            $table->json('payload')->nullable();
            $table->timestamps();
        });
    }

    public function down(): void { Schema::dropIfExists('request_logs'); }
};

```

ios-app/TreeCoursework/APIClient.swift

```
import Foundation

enum APIError: LocalizedError {
    case invalidBaseURL
    case invalidResponse
    case server(statusCode: Int, message: String)

    var errorDescription: String? {
        switch self {
            case .invalidBaseURL: return "Некорректный адрес Laravel API."
            case .invalidResponse: return "Сервер вернул неожиданный ответ."
            case let .server(statusCode, message): return "Ошибка \("\(statusCode): \("\(message)"
        }
    }
}

struct APIClient {
    private let session: URLSession = .shared

    func fetchLatestTree(baseURL: String) async throws -> TreeRecord {
        try await request(baseURL: baseURL, path: "trees/latest")
    }

    func fetchTrees(baseURL: String, limit: Int = 15) async throws -> [TreeSummary] {
        try await request(baseURL: baseURL, path: "trees?limit=\(limit)")
    }

    func fetchTree(baseURL: String, id: Int) async throws -> TreeRecord {
        try await request(baseURL: baseURL, path: "trees/\(id)")
    }

    func fetchRequestLogs(baseURL: String, limit: Int = 15) async throws -> [RequestLogEntry] {
        try await request(baseURL: baseURL, path: "request-logs?limit=\(limit)")
    }

    func generateTree(baseURL: String, payload: GenerateTreeRequest) async throws -> TreeRecord {
        let body = try makeEncoder().encode(payload)
        return try await request(baseURL: baseURL, path: "trees/generate", method: "POST", body: body)
    }

    private func request<T: Decodable>(
        baseURL: String, path: String, method: String = "GET", body: Data? = nil
```

```

) async throws -> T {
    let url = try makeURL(baseUrl: baseUrl, path: path)
    var request = URLRequest(url: url)
    request.httpMethod = method
    request.setValue("application/json", forHTTPHeaderField: "Accept")

    if let body {
        request.httpBody = body
        request.setValue("application/json", forHTTPHeaderField: "Content-Type")
    }

    let (data, response) = try await session.data(for: request)

    guard let httpResponse = response as? HTTPURLResponse else {
        throw APIError.invalidResponse
    }
    guard (200 ... 299).contains(httpResponse.statusCode) else {
        throw APIError.server(
            statusCode: httpResponse.statusCode,
            message: extractErrorMessage(from: data)
        )
    }

    let envelope = try makeDecoder().decode(APIEnvelope<T>.self, from: data)
    return envelope.data
}

private func makeDecoder() -> JSONDecoder {
    let decoder = JSONDecoder()
    decoder.keyDecodingStrategy = .convertFromSnakeCase
    decoder.dateDecodingStrategy = .iso8601
    return decoder
}

private func makeEncoder() -> JSONEncoder {
    let encoder = JSONEncoder()
    encoder.keyEncodingStrategy = .convertToSnakeCase
    return encoder
}
}

```

ios-app/TreeCoursework/AppViewModel.swift

```
import Foundation
```

```
@MainActor
```

```
final class AppViewModel: ObservableObject {
    @Published var baseUrl: String = "http://localhost:3032/api"
    @Published var titleText: String = "Сгенерированное дерево"
    @Published var orientation: TreeOrientation = .vertical

```

```

@Published var renderOrientation: TreeOrientation = .vertical
@Published var maxDepth: Int = 4
@Published var childProbability: Double = 0.6
@Published var seed: Int = 42

@Published var currentTree: TreeRecord?
@Published var treeHistory: [TreeSummary] = []
@Published var requestLogs: [RequestLogEntry] = []
@Published var selectedTreeID: Int?
@Published var isLoading: Bool = false
@Published var errorMessage: String?
@Published var infoMessage: String?

private let client = APIClient()
private var hasLoaded = false

func bootstrapIfNeeded() async {
    guard !hasLoaded else { return }
    hasLoaded = true
    await reloadAll(loadLatestTree: true)
}

func generateTree() async {
    let payload = makePayload()
    await withLoadingState {
        let tree = try await client.generateTree(baseUrl: baseUrl, payload:
payload)
        applyLoadedTree(tree)
        infoMessage = "Дерево сгенерировано и сохранено."

        async let treeHistory = client.fetchTrees(baseUrl: baseUrl)
        async let requestLogs = client.fetchRequestLogs(baseUrl: baseUrl)
        self.treeHistory = try await treeHistory
        self.requestLogs = try await requestLogs
    }
}

func loadLatestTree() async {
    await withLoadingState {
        let tree = try await client.fetchLatestTree(baseUrl: baseUrl)
        applyLoadedTree(tree)
    }
}

func loadTree(id: Int) async {
    await withLoadingState {
        let tree = try await client.fetchTree(baseUrl: baseUrl, id: id)
        applyLoadedTree(tree)
    }
}

```

```

func randomizeSeed() { seed = Int.random(in: 1 ... 999_999) }

private func makePayload() -> GenerateTreeRequest {
    let trimmedTitle = titleText.trimmingCharacters(in: .whitespacesAndNewlines)
    return GenerateTreeRequest(
        title: trimmedTitle.isEmpty ? nil : trimmedTitle,
        orientation: orientation,
        maxDepth: maxDepth,
        childProbability: childProbability,
        seed: max(1, seed)
    )
}

private func applyLoadedTree(_ tree: TreeRecord) {
    currentTree = tree
    selectedTreeID = tree.id
    renderOrientation = tree.orientation
    orientation = tree.orientation
    titleText = tree.title ?? "Сгенерированное дерево"
    maxDepth = tree.maxDepth
    childProbability = tree.childProbability
    seed = tree.seed
}

private func withLoadingState(_ action: () async throws -> Void) async {
    isLoading = true
    errorMessage = nil
    do { try await action() }
    catch {
        errorMessage = error.localizedDescription
        infoMessage = nil
    }
    isLoading = false
}
}

```

ios-app/TreeCoursework/TreeCanvasView.swift

```

import SwiftUI

struct TreeCanvasView: View {
    let root: TreeNode
    let orientation: TreeOrientation
    var useInternalScroll: Bool = true

    private let nodeSize = CGSize(width: 104, height: 72)
    private let levelSpacing: CGFloat = 88
    private let siblingSpacing: CGFloat = 30
    private let padding: CGFloat = 40

```

```

var body: some View {
    let layout = TreeLayout(
        root: root, orientation: orientation,
        nodeSize: nodeSize, levelSpacing: levelSpacing,
        siblingSpacing: siblingSpacing, padding: padding
    )

    let graph = ZStack(alignment: .topLeading) {
        ForEach(layout.edges, id: \.self) { edge in
            DirectedEdgeView(
                from: layout.point(for: edge.parentID),
                to: layout.point(for: edge.childID),
                nodeRadius: nodeCornerRadius
            )
        }
        ForEach(layout.nodes) { entry in
            TreeNodeBadge(node: entry.node).position(entry.position)
        }
    }
    .frame(width: layout.canvasSize.width, height: layout.canvasSize.height,
alignment: .topLeading)
    .padding(8)
    .background(ThemeColors.canvasBackground)

    Group {
        if useInternalScroll {
            ScrollView([.horizontal, .vertical]) { graph }
        } else {
            graph
        }
    }
}

private var nodeCornerRadius: CGFloat {
    hypot(nodeSize.width, nodeSize.height) / 2
}
}

```

// Алгоритм укладки дерева: обход в глубину, x-координаты листьев
// присваиваются последовательно (0, 1, 2, ...), x родителя – среднее
// между x первого и последнего потомка. Для горизонтальной ориентации
// оси (x, y) меняются местами.

```

private struct TreeLayout {
    struct PositionedNode: Identifiable {
        let node: TreeNode
        let position: CGPoint
        var id: Int { node.id }
    }

    struct Edge: Hashable { let parentID: Int; let childID: Int }

    let nodes: [PositionedNode]
    let edges: [Edge]
}

```



```

let canvasSize: CGSize
private let pointsByID: [Int: CGPoint]

init(root: TreeNode, orientation: TreeOrientation,
     nodeSize: CGSize, levelSpacing: CGFloat,
     siblingSpacing: CGFloat, padding: CGFloat) {
    var rawPositions: [Int: CGPoint] = [:]
    var rawNodes: [TreeNode] = []
    var edges: [Edge] = []
    var nextLeafX: CGFloat = 0
    var maxDepth = 0

    func visit(_ node: TreeNode, depth: Int) -> CGFloat {
        rawNodes.append(node)
        maxDepth = max(maxDepth, depth)
        if node.children.isEmpty {
            let leafX = nextLeafX
            nextLeafX += 1
            rawPositions[node.id] = CGPoint(x: leafX, y: CGFloat(depth))
            return leafX
        }
        let childXs = node.children.map { child -> CGFloat in
            edges.append(Edge(parentID: node.id, childID: child.id))
            return visit(child, depth: depth + 1)
        }
        let currentX = (childXs.first! + childXs.last!) / 2
        rawPositions[node.id] = CGPoint(x: currentX, y: CGFloat(depth))
        return currentX
    }

    _ = visit(root, depth: 0)

    let horizontalStep = nodeSize.width + siblingSpacing
    let verticalStep = nodeSize.height + levelSpacing
    let maxBreadth = rawPositions.values.map(\.x).max() ?? 0

    func orient(_ rawPoint: CGPoint) -> CGPoint {
        let x = padding + (rawPoint.x * horizontalStep) + (nodeSize.width / 2)
        let y = padding + (rawPoint.y * verticalStep) + (nodeSize.height / 2)
        switch orientation {
            case .vertical: return CGPoint(x: x, y: y)
            case .horizontal: return CGPoint(x: y, y: x)
        }
    }

    let pointsByID = rawPositions.mapValues(orient)
    self.pointsByID = pointsByID
    self.edges = edges
    self.nodes = rawNodes.compactMap { node in
        guard let point = pointsByID[node.id] else { return nil }
        return PositionedNode(node: node, position: point)
    }
}

```

```

    }

    let verticalCanvas = CGSize(
        width: padding * 2 + ((maxBreadth + 1) * horizontalStep) +
nodeSize.width,
        height: padding * 2 + (CGFloat(maxDepth) * verticalStep) +
nodeSize.height
    )
    switch orientation {
    case .vertical: canvasSize = verticalCanvas
    case .horizontal: canvasSize = CGSize(width: verticalCanvas.height, height:
verticalCanvas.width)
    }
}

func point(for id: Int) -> CGPoint { pointsByID[id] ?? .zero }
}

```

Вывод:

В ходе курсовой работы по дисциплине «Клиент-серверные приложения и сетевые технологии» было разработано клиент-серверное программное обеспечение, состоящее из серверной части на фреймворке Laravel 13 (PHP 8.3) и мобильного клиента на iOS (SwiftUI). Серверная часть реализует два программных модуля: первый модуль (TreeGenerationService) случайным образом формирует данные по вершинам дерева в базе данных — каждая вершина имеет до трёх дочерних узлов, наличие каждого потомка определяется псевдослучайным генератором Mt19937, инициализируемым зерном seed, что обеспечивает воспроизводимость результата; второй модуль (TreeResponseBuilder совместно с TreeController::show / latest) читает сохранённую структуру из БД и собирает её в виде вложенного JSON. Клиент на SwiftUI отображает полученное дерево в виде ориентированного графа с возможностью переключения между вертикальной и горизонтальной ориентацией (TreeCanvasView). История всех обращений к API сохраняется в таблице request_logs силами промежуточного ПО LogApiRequest, которое перехватывает каждый запрос после выполнения контроллера и фиксирует имя маршрута, HTTP-метод, путь, IP-адрес клиента, код ответа,

идентификатор связанного дерева и тело запроса. Реализация соответствует требованиям задания о разделении серверной и клиентской частей, использовании СУБД для хранения данных и журнала, и демонстрирует современный подход к построению клиент-серверных систем с REST API и мобильным клиентом.

Список используемой литературы:

1. Таненбаум Э., Уэзеролл Д. Компьютерные сети : учебник. — 6-е изд. — СПб. : Питер, 2022. — 960 с.
2. Laravel 13.x Documentation [Электронный ресурс]. — Режим доступа: <https://laravel.com/docs/13.x> (дата обращения: 18.05.2026).
3. PHP Manual. PHP 8.3 [Электронный ресурс]. — Режим доступа: <https://www.php.net/manual/ru/> (дата обращения: 18.05.2026).
4. SwiftUI Documentation. Apple Developer [Электронный ресурс]. — Режим доступа: <https://developer.apple.com/documentation/swiftui> (дата обращения: 18.05.2026).
5. PostgreSQL 16 Documentation [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/16/> (дата обращения: 18.05.2026).
6. Кнут Д.Э. Искусство программирования. Том 1. Основные алгоритмы. — 3-е изд. — М. : Вильямс, 2019. — 720 с.